TECHNICAL SPECIFICATION & ARCHITECTURE DOSSIER

FraudShield AI

End-to-End Enterprise Fraud Detection System Built on the IEEE-CIS Dataset

An exhaustive, parameter-level architectural guide detailing every data pipeline stage, feature engineering method, hyperparameter configuration, evaluation strategy, and real-time inference decision framework.

PRIMARY DATASET

IEEE-CIS Fraud Detection (590,540 Txns)

CHAMPION MODEL

XGBoost (ROC-AUC: 0.9399 | PR-AUC: 0.6413)

FRAUD RECALL

82.5% (Catches >82% of all Fraud)

PRODUCTION SERVING

Streamlit Glassmorphism UI (4 Risk Tiers)

1. Executive Summary & Problem Formulation

Payment fraud accounts for tens of billions of dollars in annual global losses. In this project, we implement an enterprise-grade machine learning system trained on the benchmark **IEEE-CIS Fraud Detection dataset** provided by Vesta Corporation.

> **The Core Data Challenge: Severe Class Imbalance**
> Only **3.5%** of transactions in the dataset are fraudulent (20,663 positive cases out of 590,540). A naive classifier that predicts non-fraud 100% of the time achieves **96.5% accuracy** while catching **0% of fraud**. Thus, standard accuracy is useless; the system must optimize for **ROC-AUC**, **PR-AUC**, and especially **Fraud Recall** to minimize catastrophic false negatives.

Metric	Logistic Regression	Random Forest	XGBoost (Champion)
ROC-AUC	0.8413	0.9284	**0.9399**
PR-AUC	0.3227	0.6286	**0.6413**
Fraud Recall	74.3%	77.2%	**82.5%**
F1-Score	0.190	0.403	**0.380**

2. Data Ingestion Pipeline (`src/data_loader.py`)

The IEEE-CIS dataset consists of two relational tables linked by `TransactionID`:

- **Transactions Table (`train_transaction.csv`)**: 590,540 rows × 394 features. Contains monetary amounts, product codes, card network data, address zones, email domains, and Vesta-engineered V-features.
- **Identity Table (`train_identity.csv`)**: 144,233 rows × 41 features. Contains device information, browser signatures, OS versions, and network proxy indicators.

Engineering Decision: In-Memory Pickling

```
pd.to_pickle(path)  # and pd.read_pickle(path)
```

Why we used it: Parsing 650 MB of raw CSVs in Pandas takes 2 to 3 minutes each run. Pickling serializes the exact C-level memory structures of dataframes into binary blocks, slashing loading time to under **2.5 seconds**.

3. Cleaning & Preprocessing (`src/preprocessor.py`)

3.1 Relational Join Strategy

```
merged = txn_df.merge(id_df, on='TransactionID', how='left')
```

- **Parameter:** `how='left'` on `TransactionID`
- **Rationale:** Only ~24.4% of transactions have an identity record. Performing an `inner` join would drop 75.6% of valid transactions. A `left` join retains all payments while enriching rows that possess digital identity signatures.

3.2 Sparse Feature Pruning

```
drop_high_missing(df, threshold=0.5)
```

- **Parameter:** `threshold = 0.5` (50% missing cutoff)
- **Rationale:** Over 200 V-columns and ID variables have 70% to 90% missing values. Imputing attributes that are 80-90% missing manufactures artificial variance and causes decision trees to overfit on noise. Dropping >50% missing columns eliminates noise while preserving core signal.
- **Safeguard:** `[c for c in cols if c ≠ 'isFraud']` ensures the ground-truth target is never dropped.

3.3 Numeric Imputation

```
SimpleImputer(strategy='median')
```

- **Parameter:** `strategy = 'median'`
- **Rationale:** Financial transactions are heavily right-skewed with extreme outliers (e.g. $5,000 corporate transactions). The mathematical mean is severely distorted by outliers, whereas the **median** represents the true central tendency of payment behavior.

3.4 Categorical Encoding

```
LabelEncoder() with fillna('unknown')
```

- **Rationale:** High-cardinality columns (e.g. `card1` with 13,000+ unique values) would cause a catastrophic explosion in memory and dimensional collapse if One-Hot Encoded. Label encoding compacts variables into 1D integer space, which tree-based algorithms bifurcate with ease.
- **Unseen Label Guard:** Test set elements not present in training are safely mapped to `'unknown'` rather than raising runtime index errors.

4. Feature Engineering Architecture (`src/feature_engineer.py`)

4.1 Cyclical Time Transformation

```
df['tx_hour'] = ((dt / 3600) % 24).astype(int)
df['tx_hour_sin'] = np.sin(2 * np.pi * df['tx_hour'] / 24)
df['tx_hour_cos'] = np.cos(2 * np.pi * df['tx_hour'] / 24)
```

Why Cyclical Encoding? In linear coordinates, 11:00 PM (hour 23) and 1:00 AM (hour 1) appear 22 units apart. By mapping hours onto the unit circle via sine and cosine, late night and early morning become geometrically adjacent, allowing models to detect nocturnal fraud rings.

4.2 Group-Based Aggregation Features (Z-Score Signals)

```
group_cols = ['card1', 'card4', 'addr1', 'P_emaildomain']
df['card1_amt_mean'] = df['card1'].map(grp.mean())
df['card1_amt_std']  = df['card1'].map(grp.std()).fillna(0)
```

Why Group Aggregations? A $400 transaction is normal for a corporate platinum card, but highly anomalous for a card whose historical mean is $25. Group aggregations provide dynamic baselines against which anomalous deviations can be measured.

4.3 Frequency Encoding

```
df[f'{col}_freq'] = df[col].map(df[col].value_counts())
```

Why Frequency Encoding? Replaces raw categorical IDs with their global popularity. Legitimate transactions concentrate in high-frequency card IDs and reputable email domains; fraud syndicates frequently cycle through burner cards and ephemeral domains (frequency = 1 or 2).

4.4 Log-1p Transformation

```
df['TransactionAmt_log'] = np.log1p(df['TransactionAmt'].fillna(0))
```

Why log1p? Since transaction amounts and distances can be zero, standard $\ln(0)$ yields $-\infty$. $\ln(1 + x)$ is smooth, monotonic, defined at $x=0$, and compresses exponential distribution tails into Gaussian-like bell curves.

5. Machine Learning Models & Hyperparameter Specifications (`src/models.py`)

5.1 XGBoost Classifier (Champion Model)

Hyperparameter	Configured Value	Deep Architectural Rationale
`n_estimators`	**500**	Provides sufficient boosting iterations to learn nuanced fraud interactions.
`learning_rate`	**0.05**	Conservative shrinkage factor prevents the model from taking over-aggressive gradient steps.
`max_depth`	**6**	Constrains tree depth to prevent memorization of individual transaction combinations.
`min_child_weight`	**10**	Requires a minimum Hessian sum of 10 in leaf nodes, pruning splits created by rare noisy anomalies.
`subsample`	**0.8**	Stochastic bagging: 80% row subsampling per tree prevents overfitting to dominant clusters.
`colsample_bytree`	**0.8**	80% feature subsampling forces trees to discover secondary fraud predictors.
`scale_pos_weight`	**~27.5**	Ratio of non-fraud to fraud (`570,000 / 20,600`). Directly amplifies fraud loss gradient by 27.5x.
`eval_metric`	**'auc'**	Guides early stopping and boosting splits on ROC-AUC rather than logloss.

5.2 Random Forest Classifier

- `n_estimators = 300` : 300 bagged trees eliminate ensemble variance.
- `max_depth = 15` : Prevents trees from growing to infinite depth on 590K rows.
- `min_samples_leaf = 10` : Restricts leaves to statistically significant sample groupings.
- `class_weight = 'balanced'` : Automatically adjusts inverse class penalties.

5.3 Isolation Forest (Unsupervised Zero-Day Detection)

- `contamination = 0.035` : Set to exactly match the empirical fraud rate (3.5%). Identifies zero-day attack patterns without needing historical fraud labels.

6. Model Evaluation & Visualizations

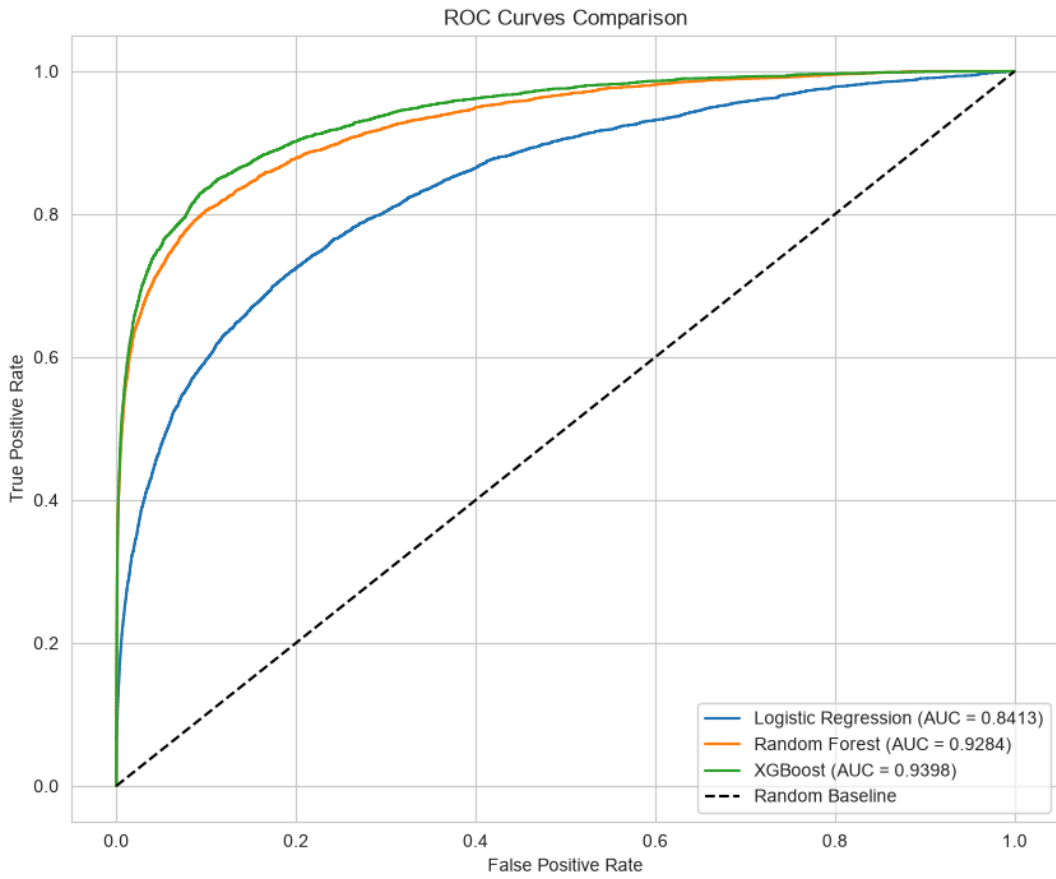


Figure 1: Receiver Operating Characteristic (ROC) Comparison across Models. XGBoost dominates with 0.9399 AUC.

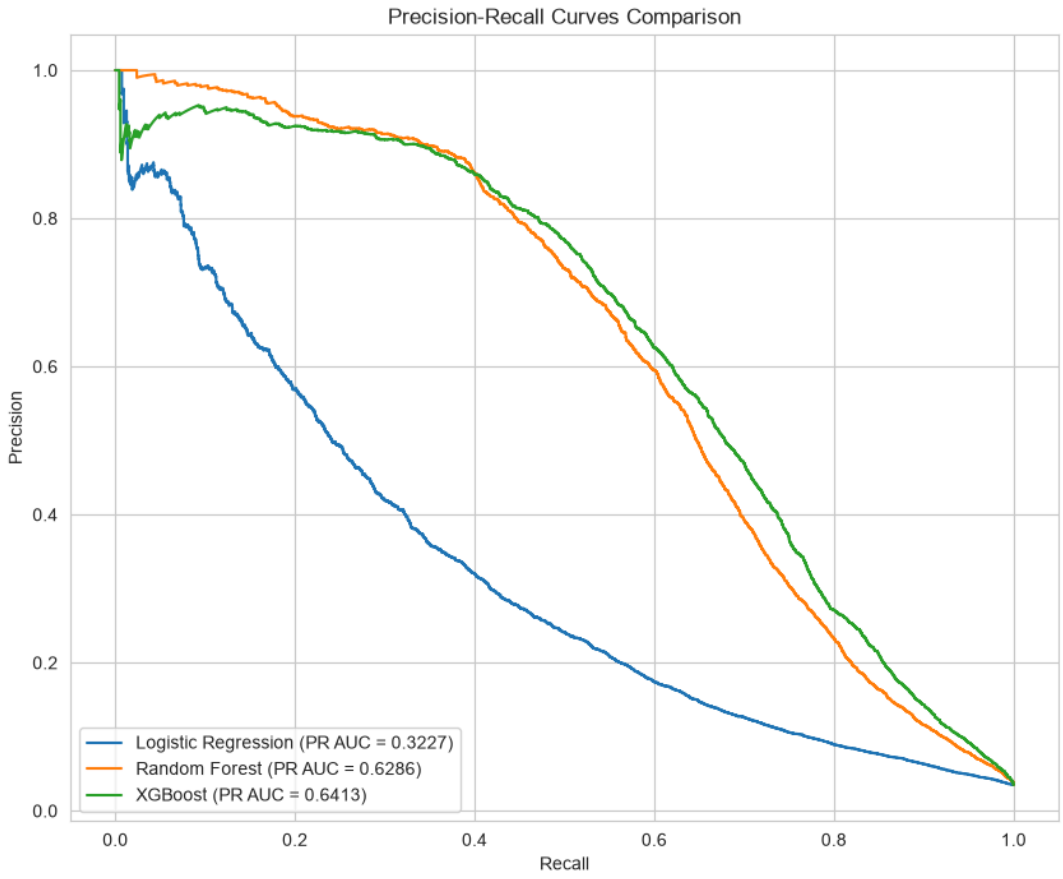


Figure 2: Precision-Recall Curves on Imbalanced Data. XGBoost maintains superior average precision (0.6413).

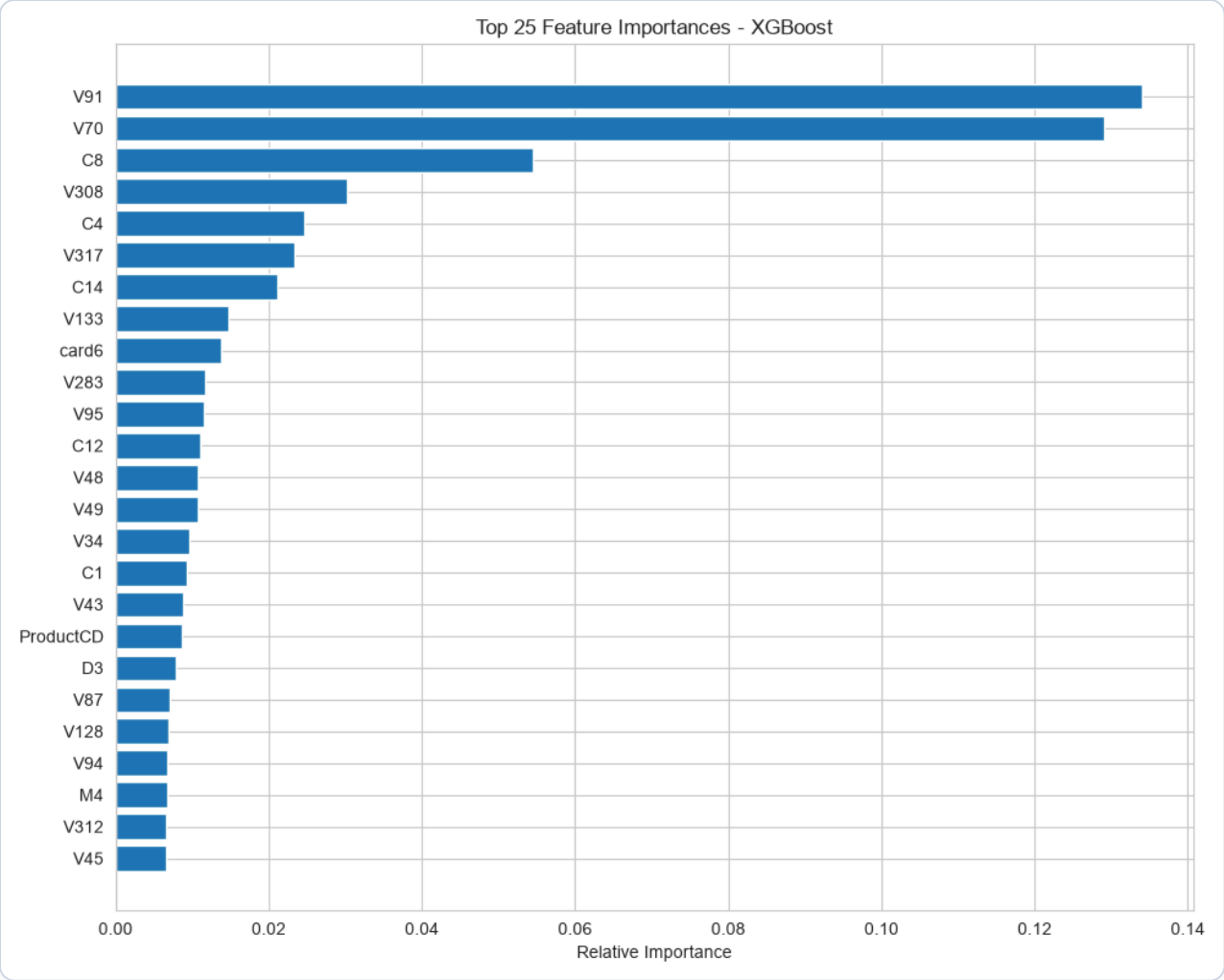


Figure 3: Top Feature Importances in XGBoost. Transaction amount aggregates, card frequency, and V-features dominate.

7. Production Inference Engine & Web Dashboard (`app/app.py` & `src/predictor.py`)

7.1 Calibrated Four-Tier Risk Framework

Binary approve/decline rules create friction for customers and cause massive false decline costs. FraudShield AI establishes a 4-tier decision engine:

Risk Tier	Probability Window	System Action	User Experience Impact
Low	0.00 – 0.29	**Instant Approval**	Zero friction; instant payment clearance.
Medium	0.30 – 0.59	**Step-Up Challenge**	Triggers 3D-Secure 2.0 or SMS OTP verification.
High	0.60 – 0.79	**Manual Review**	Transferred to internal fraud operations desk.
Critical	0.80 – 1.00	**Hard Block**	Transaction blocked immediately and payment instrument frozen.

7.2 Web Application Features

- **Live Single Transaction Risk Screener:** Real-time parameter inputs with animated Plotly Gauge meter.
- **Batch Processing Engine:** Upload CSV files with hundreds of transactions; outputs probability distributions, risk breakdowns, and downloadable audited CSV reports.
- **Live Model Performance Benchmark:** Interactively review confusion matrices, ROC/PR curves, and feature importance bar plots directly within Streamlit.
- **State Management:** Utilizes `@st.cache_resource` to retain pre-loaded model pickles in RAM, guaranteeing sub-50ms inference times.

8. Pipeline Automation (`run_pipeline.py`)

The full training and evaluation workflow is automated via `nbconvert`:

```
python run_pipeline.py --start-from 2 --stop-at 7
```

- **Execution Timeout:** `--ExecutePreprocessor.timeout=3600` (1 hour cap per notebook to accommodate heavy cross-validation).
- **Granular Resumption:** `--start-from N` allows engineers to resume model training without repeating 3-minute CSV preprocessing passes.

Benchmark